PELICULERAS

MEMORIA TÉCNICA

23 de marzo de 2026

Miren Mailharin e Iratxe Lasa

1. INTRODUCCIÓN Y OBJETIVOS

El proyecto Peliculeras consiste en el desarrollo integral de una plataforma de vídeo bajo demanda (VOD). Partiendo de un entorno de React desde cero, el objetivo principal ha sido crear una interfaz dinámica y funcional que permita a los cinéfilos interactuar con un catálogo, gestionar su biblioteca personal y participar en una comunidad activa mediante reseñas y puntuaciones.

Despliegue hosting: https://dsm-catalogo.web.app/

Repositorio github: https://github.com/mireeen/dsm-webapp.git

2. ARQUITECTURA DEL SISTEMA

La aplicación sigue un modelo de componentes funcionales y una jerarquía clara, diseñada para maximizar la reutilización de código y la eficiencia en el renderizado. A continuación, se detalla la estructura principal:

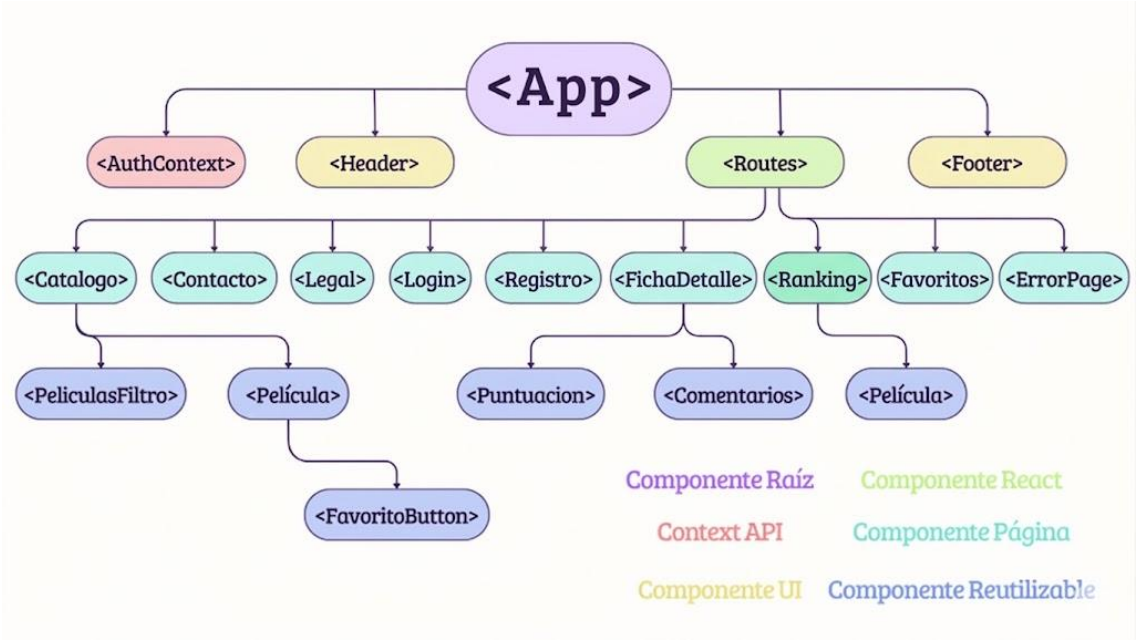

Figura 1: Esquema de la Arquitectura

Como se observa en el diagrama, el proyecto se organiza de la siguiente manera:

- **Componente Raíz (*App*):** Orquestador principal que envuelve toda la lógica del proyecto, definiendo las rutas dinámicas.

- **Context API (*AuthProvider*):** Proporciona las credenciales de autenticación de forma transversal, permitiendo gestionar el estado del usuario en toda la aplicación.

- **Componentes UI (*Header* y *Footer*):** Componentes transversales que mantienen la consistencia visual en todas las vistas.

- **Sistema de Rutas (*React Routes*):** Se utiliza para la navegación multi-página sin recarga del navegador. Destaca el uso de parámetros dinámicos para renderizar fichas específicas de forma eficiente.
- **Componentes de Página:** Vistas principales como *Catalogo*, *Contacto*, *Legal*, *Login*, *Registro*, *FichaDetalle*, *Favoritos, Ranking* y *ErrorPage*.

- **Componentes Reutilizables:** Elementos que se emplean en distintas partes de la aplicación para mantener la consistencia como P*eliculasFiltro*, *Pelicula, Puntuacion*, *FavoritoButton* y *Comentarios*.

3. EXPERIENCIA DE USUARIO Y FUNCIONALIDADES

3.1. Navegación e Interacción

El núcleo de *Peliculeras* reside en un catálogo dinámico de contenido VOD diseñado para que el usuario navegue de forma intuitiva por la biblioteca de títulos. Para lograr esta fluidez, se ha implementado un sistema de filtrado segmentado por categoría o género, que organiza las películas de manera instantánea, optimizando la experiencia de usuario. Cada título se presenta mediante tarjetas visuales reutilizables que muestran información esencial, como la puntuación media y el acceso directo a la gestión de favoritos.

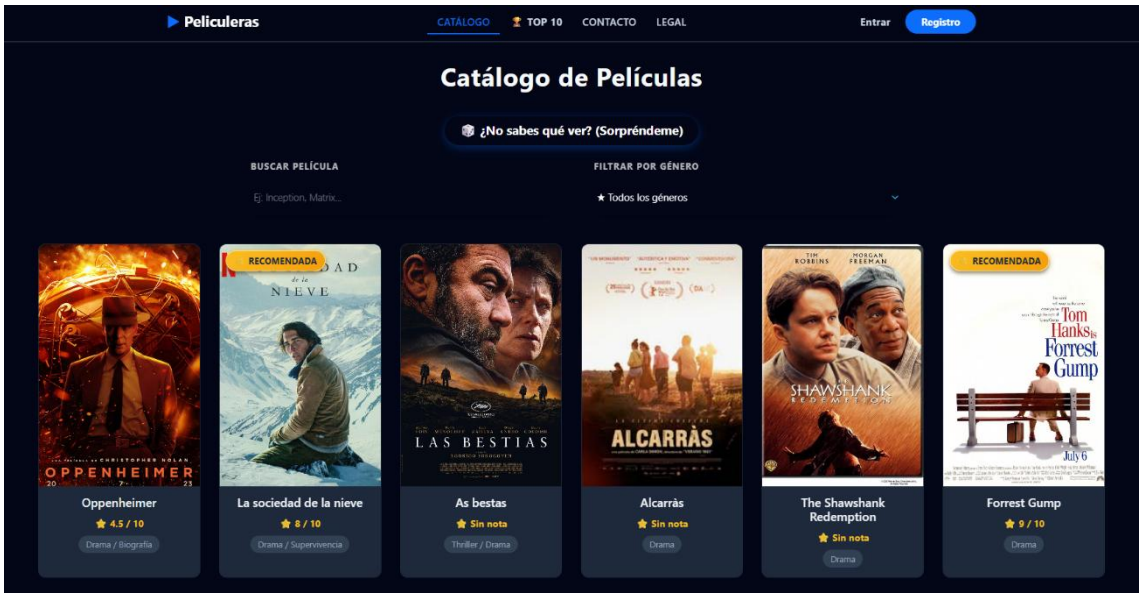

Figura 2: Página Principal

Al interactuar con cualquier elemento del catálogo, el sistema emplea parámetros dinámicos en la URL para redirigir al usuario a una ficha de detalle específica. En esta vista, la aplicación realiza llamadas asíncronas a la base de datos de Firebase para recuperar y calcular en tiempo real las valoraciones de la comunidad. Además, para fomentar la interactividad, los usuarios registrados disponen de un hilo de comentarios donde pueden publicar sus propias reseñas y leer las impresiones de otros suscriptores a través de un formulario integrado.

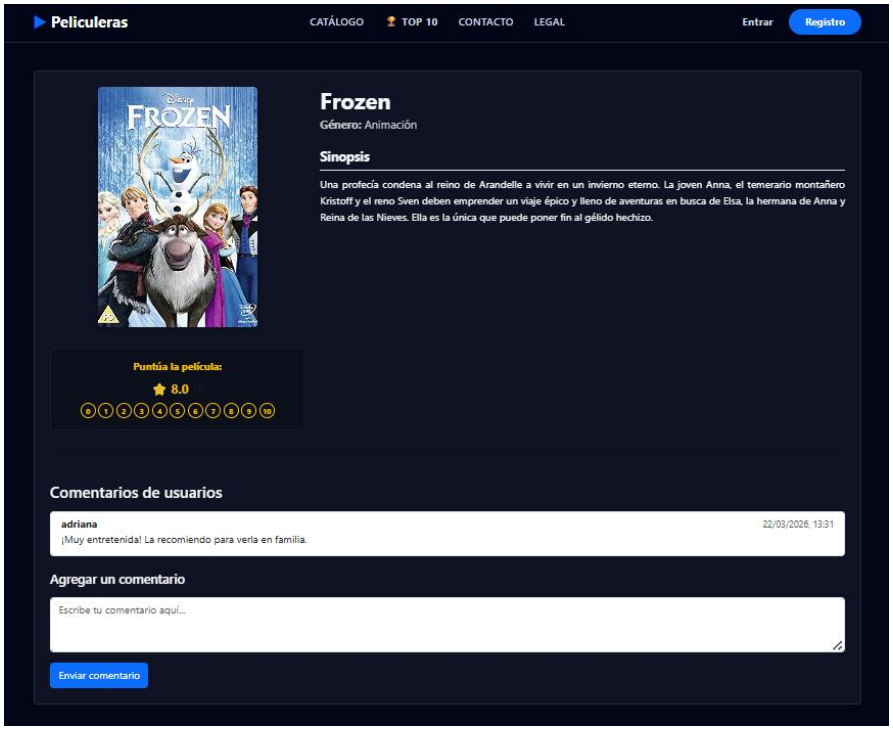

Figura 3: Ficha de Detalle

3.2. Funcionalidades Avanzadas

Con el objetivo de aportar un valor añadido al proyecto, se han desarrollado tres herramientas que mejoran significativamente la usabilidad y el “engagement”. En primer lugar, se implementó un buscador inteligente que filtra el catálogo en tiempo real por título, una solución eficaz para evitar el "scroll" infinito y localizar contenido rápidamente. Complementando esta búsqueda, el algoritmo de Recomendación Aleatoria ("Sorpréndeme") ofrece una alternativa dinámica para aquellos usuarios indecisos, seleccionando y redirigiendo automáticamente a una ficha de película al azar mediante una consulta a la base de datos.

Finalmente, la robustez de la webapp se completa con el Ranking TOP 10, una sección dedicada que procesa las puntuaciones globales de la comunidad para destacar los títulos mejor valorados. Esta funcionalidad no solo sirve como guía de calidad, sino que incentiva la participación de los usuarios para influir en el posicionamiento de sus películas favoritas. Todo este ecosistema se apoya en una gestión avanzada de la biblioteca personal, permitiendo a cada espectador construir su propia lista de favoritos para consultas posteriores.

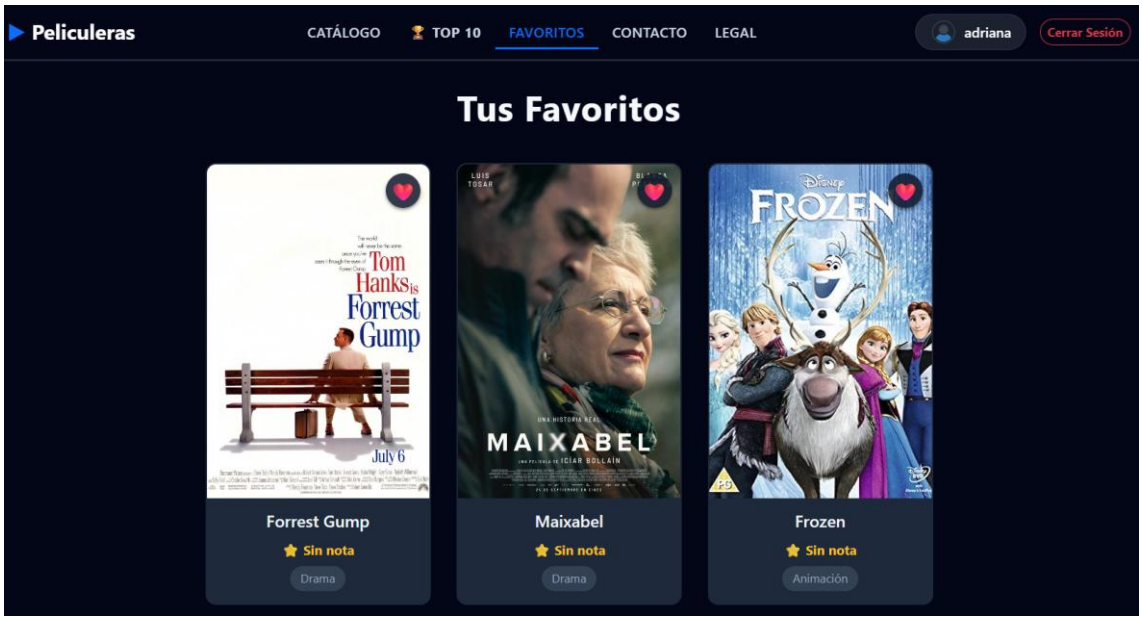

Figura 4: Favoritos

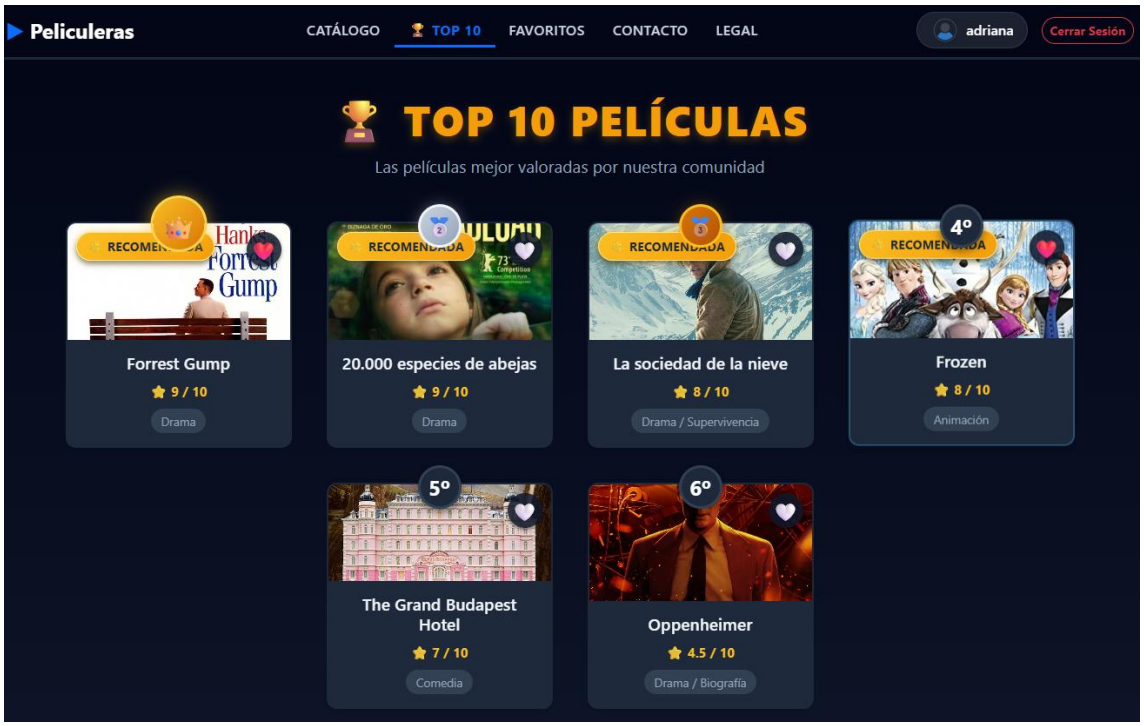

Figura 5: Top 10

4. TECNOLOGÍAS Y LIBRERÍAS UTILIZADAS

Librerías y herramientas utilizadas

React :
Librería principal de JavaScript utilizada para construir la interfaz de usuario basada en componentes reutilizables.

Vite:
Herramienta de desarrollo y compilación que actúa como *bundler* y servidor, proporcionando un entorno rápido y eficiente, sustituyendo a Create React App.

React Router:
Librería encargada de la navegación dentro de la aplicación, permitiendo un comportamiento de *Single Page Application (SPA)* sin recargas completas de página.

Diseño e interfaz de usuario

Bootstrap: Framework CSS utilizado para diseñar interfaces responsive y estructurar la aplicación mediante su sistema de rejilla y componentes predefinidos.

React-Bootstrap:
Implementación de Bootstrap adaptada a React, que permite usar componentes como botones, formularios y modales de forma más integrada.

CSS puro:
Hojas de estilo personalizadas para aplicar ajustes específicos de diseño, como animaciones, estilos avanzados o efectos visuales personalizados.

Comunicación y backend

Axios:
Cliente HTTP basado en promesas utilizado para realizar peticiones al backend, como consultas y actualizaciones de datos.

Firebase Realtime Database:
Base de datos en tiempo real utilizada para almacenar información como el catálogo de películas, comentarios y valoraciones.

Firebase Authentication:
Servicio de autenticación que permite gestionar usuarios, incluyendo registro, inicio de sesión y control de acceso mediante tokens.

Firebase Hosting:
Plataforma utilizada para desplegar y alojar la aplicación web en internet.

5. METODOLOGÍA DE TRABAJO

El desarrollo se ha realizado de forma colaborativa entre las dos participantes de la pareja, utilizando un flujo de trabajo basado en ramas (branches) en GitHub. El historial de commits refleja una evolución constante desde el inicio del proyecto, cumpliendo con la exigencia de transparencia en el desarrollo.